THE UNIVERSITY OF AZAD JAMMU AND KASHMIR MUZAFFARABAD



Department of Software Engineering

Subject: Software Testing Lab
Lab Instructor: Engr. Fahad Nisar

Semester: 5th SET

Session: 2024 – 28

Lab Task: 4

Name: Syed Jaffar Mukhtar
Roll No: 2024 – SET – 19

1. Objective

The aim of this lab is to build a small taxi-fare module using only the six written rules (R1 to R6), without copying a reference implementation, and to verify each rule with Node.js unit tests using the built-in node:test runner. Every test follows the Arrange–Act–Assert structure. The totalDistance function (R6) was developed test-first: its tests were run while the function was unimplemented (red), then the implementation was added and the tests passed (green).

2. Environment and Project Setup

Item	Details
Runtime	Node.js v22 (v20 or newer is required)
Test runner	node:test with node:assert/strict, no third-party packages
Module type	ES modules ("type": "module" in package.json)
Test command	npm test, which runs node --test
Project layout	src/taxi.js, test/taxi.test.js, package.json

Set-up commands used:

```
npm init -y
npm pkg set type=module
npm pkg set scripts.test="node --test"
mkdir src test
```

3. Written Rules

Rule	Description
R1	fareForRide(distanceKm, ratePerKm) returns distanceKm × ratePerKm rounded to the nearest whole rupee.
R2	totalFare(rides) adds up the R1 fare of every ride. No rides gives 0.
R3	applyPeakSurcharge(amount, percent) raises amount by percent and rounds to the nearest whole rupee.
R4	waitingCharge(minutes): the first 5 minutes are free, each further minute costs Rs 10.
R5	rideSummary(rides, surchargePercent, minutesWaited) returns { fare, surcharge, waitingCharge, total }.
R6	totalDistance(rides) returns the sum of distanceKm over all rides. No rides gives 0.

4. Implementation (src/taxi.js)

Each function was written from its rule alone:

- **R1:** a single Math.round of distance times rate.
- **R2:** reduce over the rides, calling fareForRide for each, so every ride is rounded on its own as R1 demands; the start value 0 handles an empty list.
- **R3:** Math.round(amount * (100 + percent) / 100).
- **R4:** if minutes is above 5 the chargeable minutes are minutes minus 5, otherwise 0; the result is multiplied by 10.
- **R5:** surcharge is the surcharged fare minus the plain fare; total is fare + surcharge + waiting.
- **R6:** reduce over the rides adding distanceKm, starting at 0.

```
// src/taxi.js
export function fareForRide(distanceKm, ratePerKm) {
  return Math.round(distanceKm * ratePerKm);
```

```
}

export function totalFare(rides) {
  return rides.reduce((sum, r) => sum + fareForRide(r.distanceKm, r.ratePerKm), 0);
}

export function applyPeakSurcharge(amount, percent) {
  return Math.round((amount * (100 + percent)) / 100);
}

export function waitingCharge(minutes) {
  const chargeable = minutes > 5 ? minutes - 5 : 0;
  return chargeable * 10;
}

export function rideSummary(rides, surchargePercent, minutesWaited) {
  const fare = totalFare(rides);
  const surcharge = applyPeakSurcharge(fare, surchargePercent) - fare;
  const waiting = waitingCharge(minutesWaited);
  return { fare, surcharge, waitingCharge: waiting, total: fare + surcharge + waiting };
}

export function totalDistance(rides) {
  return rides.reduce((sum, r) => sum + r.distanceKm, 0);
}
```

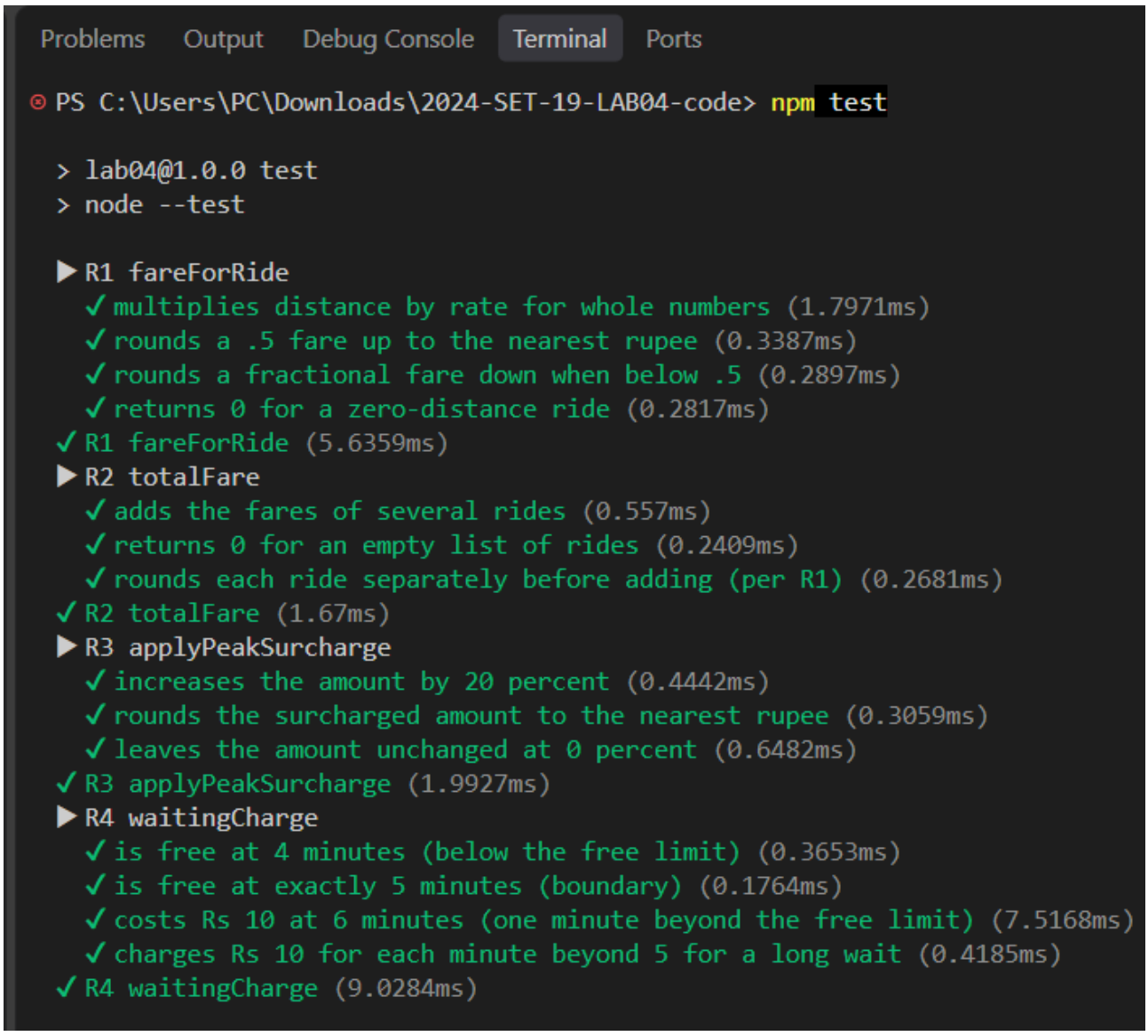
5. Test Strategy

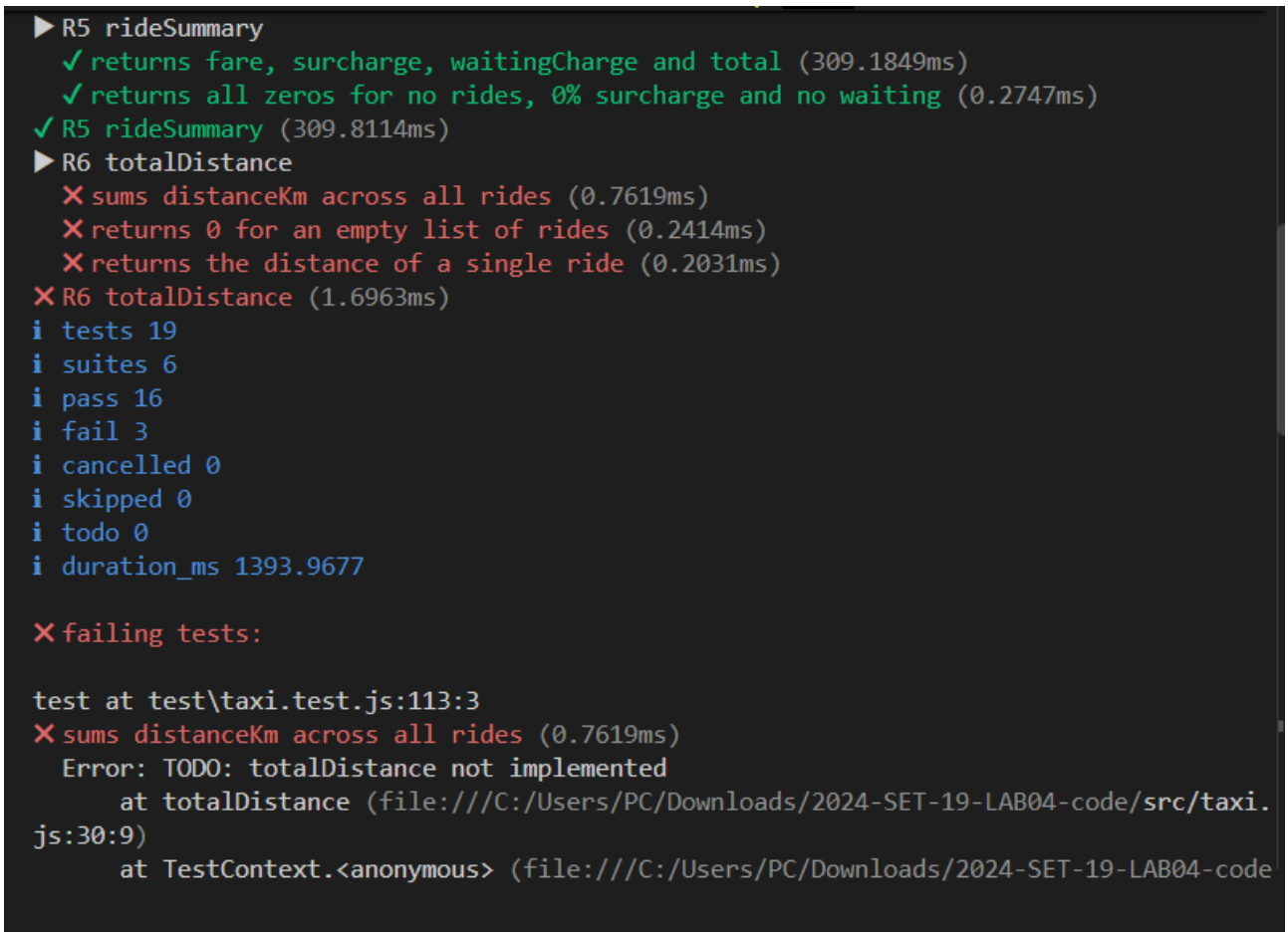
- A total of 19 tests are organised into six describe blocks, one for each rule R1–R6. This exceeds the minimum requirement of 12 tests.
- Each test is named after the behaviour it checks and uses Arrange–Act–Assert reasoning.
- Expected values are calculated by hand from the written rules.
- R4 includes all required boundary values: 4, 5 and 6 minutes, plus a longer 15-minute wait.
- Additional edge cases include empty ride lists, zero distance, zero-percent surcharge, rounding at .5, rounding below .5, and per-ride rounding in R2.
- R6 was implemented test-first: the three R6 tests failed before implementation and passed after implementation.

6. Test-First Evidence for totalDistance (R6)

6.1 Red phase: totalDistance not yet implemented

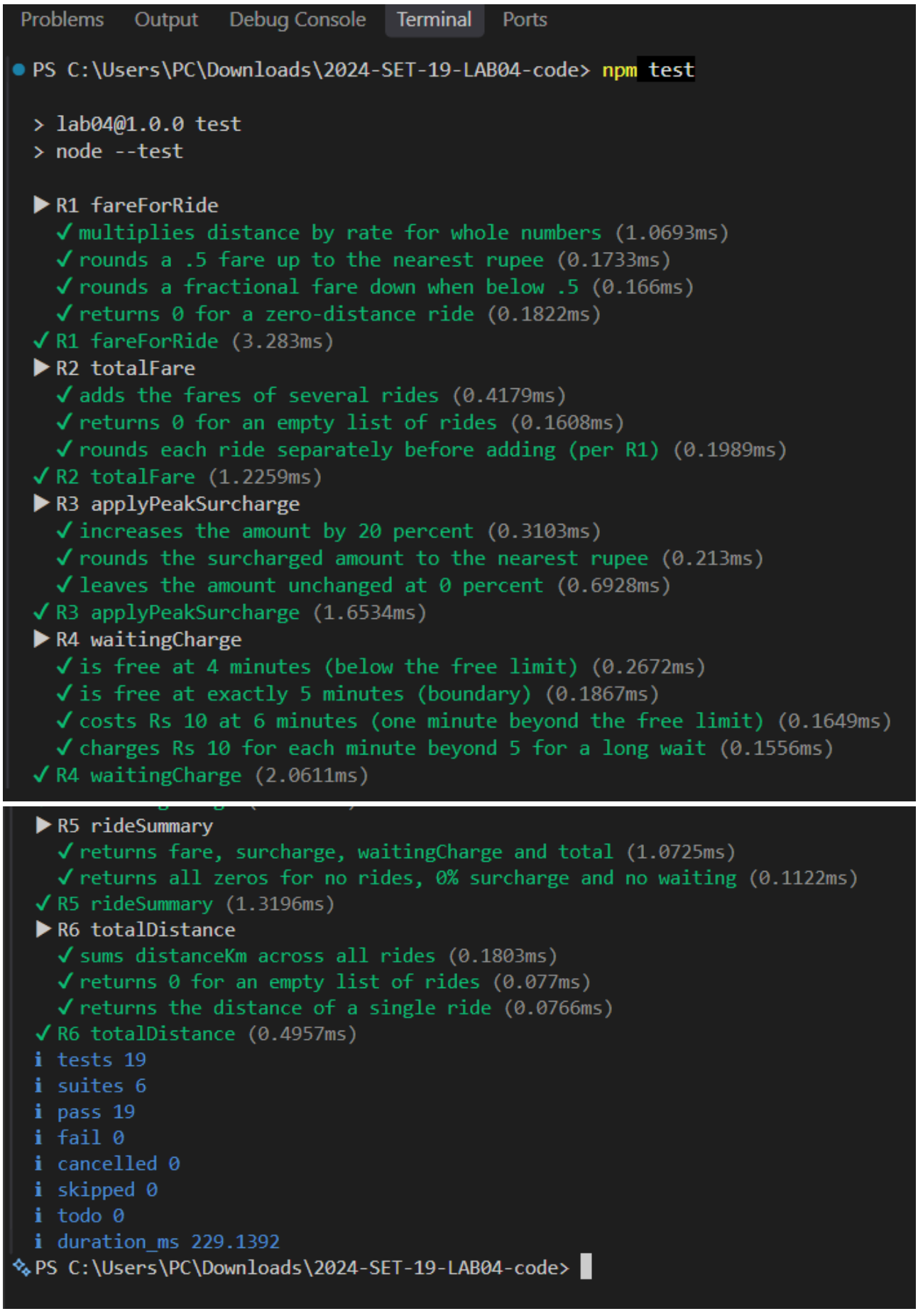
The red terminal evidence below is the 19-test run. The first part shows R1–R4 passing; the continuation shows R5 passing and all three R6 tests failing because totalDistance was not yet implemented. The run summary is 19 tests, 16 passed and 3 failed.


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\PC\Downloads\2024-SET-19-LAB04-code> npm test
> lab04@1.0.0 test
> node --test
▶ R1 fareForRide
✓ multiplies distance by rate for whole numbers (1.7971ms)
✓ rounds a .5 fare up to the nearest rupee (0.3387ms)
✓ rounds a fractional fare down when below .5 (0.2897ms)
✓ returns 0 for a zero-distance ride (0.2817ms)
✓ R1 fareForRide (5.6359ms)
▶ R2 totalFare
✓ adds the fares of several rides (0.557ms)
✓ returns 0 for an empty list of rides (0.2409ms)
✓ rounds each ride separately before adding (per R1) (0.2681ms)
✓ R2 totalFare (1.67ms)
▶ R3 applyPeakSurcharge
✓ increases the amount by 20 percent (0.4442ms)
✓ rounds the surcharged amount to the nearest rupee (0.3059ms)
✓ leaves the amount unchanged at 0 percent (0.6482ms)
✓ R3 applyPeakSurcharge (1.9927ms)
▶ R4 waitingCharge
✓ is free at 4 minutes (below the free limit) (0.3653ms)
✓ is free at exactly 5 minutes (boundary) (0.1764ms)
✓ costs Rs 10 at 6 minutes (one minute beyond the free limit) (7.5168ms)
✓ charges Rs 10 for each minute beyond 5 for a long wait (0.4185ms)
✓ R4 waitingCharge (9.0284ms)



6.2 Green phase: totalDistance implemented

After implementing totalDistance, the green terminal evidence shows the complete 19-test suite passing: 19 passed and 0 failed. This screenshot matches the 19-test suite documented in the report.


Problems
Output
Debug Console
Terminal
Ports
PS C:\Users\PC\Downloads\2024-SET-19-LAB04-code> npm test
> lab04@1.0.0 test
> node --test
▶ R1 fareForRide
✓ multiplies distance by rate for whole numbers (1.0693ms)
✓ rounds a .5 fare up to the nearest rupee (0.1733ms)
✓ rounds a fractional fare down when below .5 (0.166ms)
✓ returns 0 for a zero-distance ride (0.1822ms)
✓ R1 fareForRide (3.283ms)
▶ R2 totalFare
✓ adds the fares of several rides (0.4179ms)
✓ returns 0 for an empty list of rides (0.1608ms)
✓ rounds each ride separately before adding (per R1) (0.1989ms)
✓ R2 totalFare (1.2259ms)
▶ R3 applyPeakSurcharge
✓ increases the amount by 20 percent (0.3103ms)
✓ rounds the surcharged amount to the nearest rupee (0.213ms)
✓ leaves the amount unchanged at 0 percent (0.6928ms)
✓ R3 applyPeakSurcharge (1.6534ms)
▶ R4 waitingCharge
✓ is free at 4 minutes (below the free limit) (0.2672ms)
✓ is free at exactly 5 minutes (boundary) (0.1867ms)
✓ costs Rs 10 at 6 minutes (one minute beyond the free limit) (0.1649ms)
✓ charges Rs 10 for each minute beyond 5 for a long wait (0.1556ms)
✓ R4 waitingCharge (2.0611ms)
▶ R5 rideSummary
✓ returns fare, surcharge, waitingCharge and total (1.0725ms)
✓ returns all zeros for no rides, 0% surcharge and no waiting (0.1122ms)
✓ R5 rideSummary (1.3196ms)
▶ R6 totalDistance
✓ sums distanceKm across all rides (0.1803ms)
✓ returns 0 for an empty list of rides (0.077ms)
✓ returns the distance of a single ride (0.0766ms)
✓ R6 totalDistance (0.4957ms)
ℹ tests 19
ℹ suites 6
ℹ pass 19
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 229.1392
PS C:\Users\PC\Downloads\2024-SET-19-LAB04-code>

7. Execution Summary

Rule	Function	Tests	Passed	Failed	Status
R1	fareForRide	4	4	0	Pass
R2	totalFare	3	3	0	Pass
R3	applyPeakSurcharge	3	3	0	Pass
R4	waitingCharge	4	4	0	Pass
R5	rideSummary	2	2	0	Pass
R6	totalDistance	3	3	0	Pass
Total		19	19	0	Pass

Execution result: 19 tests are defined across six describe blocks. The green screenshot records 19 passing tests and zero failures. The red screenshot records the test-first state with 16 passing tests and the three R6 tests failing before the implementation was added.

8. Conclusion

The taxi-fare module covers all six written rules and is verified by 19 automated unit tests. The required waitingCharge boundary cases (4, 5 and 6 minutes) are included. The red/green evidence demonstrates the test-first workflow for totalDistance: three R6 tests failed before implementation and passed after implementation.

9. Unit Test Record Table:

The following table records each of the 19 tests shown in test/taxi.test.js. Expected values are calculated from the written rules. Where a test contains more than one assertion, both expected results are recorded.

Rule	Exact test name	Arrange	Act	Assert (hand-calculated)	Result	Evidence
R1	multiplies distance by rate for whole numbers	10 km at Rs 30/km	fareForRide(10, 30)	300 (10 × 30); strictEqual	Pass	Green
R1	rounds a .5 fare up to the nearest rupee	7.5 km at Rs 25/km	fareForRide(7.5, 25)	188 (187.5 rounds up); strictEqual	Pass	Green
R1	rounds a fractional fare down when below .5	3.3 × Rs 10; also 2.2 × Rs 11	fareForRide(3.3, 10); fareForRide(2.2, 11)	33 and 24; strictEqual (two assertions)	Pass	Green
R1	returns 0 for a zero-distance ride	0 km at Rs 30/km	fareForRide(0, 30)	0; strictEqual	Pass	Green
R2	adds the fares of several rides	10 km @30 and 5 km @40	totalFare(rides)	500 (300 + 200); strictEqual	Pass	Green
R2	returns 0 for an empty list of rides	Empty ride list	totalFare([])	0; strictEqual	Pass	Green
R2	rounds each ride separately before adding (per R1)	Two rides: 2.5 km @25 each	totalFare(rides)	126 (63 + 63); strictEqual	Pass	Green

R3	increases the amount by 20 percent	Rs 1000; 20% surcharge	applyPeakSurcharge(1000, 20)	1200; strictEqual	Pass	Green
R3	rounds the surcharged amount to the nearest rupee	Rs 333; 10% surcharge	applyPeakSurcharge(333, 10)	366 (366.3 rounds to 366); strictEqual	Pass	Green
R3	leaves the amount unchanged at 0 percent	Rs 450; 0% surcharge	applyPeakSurcharge(450, 0)	450; strictEqual	Pass	Green
R4	is free at 4 minutes (below the free limit)	4 minutes waited	waitingCharge(4)	0; strictEqual	Pass	Green
R4	is free at exactly 5 minutes (boundary)	5 minutes waited	waitingCharge(5)	0; strictEqual	Pass	Green
R4	costs Rs 10 at 6 minutes (one minute beyond the free limit)	6 minutes waited	waitingCharge(6)	10; strictEqual	Pass	Green
R4	charges Rs 10 for each minute beyond 5 for a long wait	15 minutes waited	waitingCharge(15)	100 ((15 − 5) × 10); strictEqual	Pass	Green
R5	returns fare, surcharge, waitingCharge and total	10 km @30; 5 km @40; 10% surcharge; 8 min wait	rideSummary(rides, 10, 8)	{fare:500, surcharge:50, waitingCharge:30, total:580}; deepStrictEqual	Pass	Green
R5	returns all zeros for no rides, 0% surcharge and no waiting	No rides; 0%; 0 minutes	rideSummary([], 0, 0)	{fare:0, surcharge:0, waitingCharge:0, total:0}; deepStrictEqual	Pass	Green
R6	sums distanceKm across all rides	10 km, 5.5 km and 4.5 km	totalDistance(rides)	20 (10 + 5.5 + 4.5); strictEqual	Pass	Red → Green
R6	returns 0 for an empty list of rides	Empty ride list	totalDistance([])	0; strictEqual	Pass	Red → Green
R6	returns the distance of a single ride	One ride of 7 km	totalDistance([{ distanceKm: 7 }])	7; strictEqual	Pass	Red → Green

10. Appendix A: Test Code (test/taxi.test.js)

```
// src/taxi.js
export function fareForRide(distanceKm, ratePerKm) {
  return Math.round(distanceKm * ratePerKm);
}


export function totalFare(rides) {
  let sum = 0;
  for (const ride of rides) {
    sum += fareForRide(ride.distanceKm, ride.ratePerKm);
  }
  return sum;
}


export function applyPeakSurcharge(amount, percent) {
  return Math.round((amount * (100 + percent)) / 100);
}


export function waitingCharge(minutes) {
  return Math.max(0, minutes - 5) * 10;
}


export function rideSummary(rides, surchargePercent, minutesWaited) {
  const fare = totalFare(rides);
  const surcharge = applyPeakSurcharge(fare, surchargePercent) - fare;
  const waiting = waitingCharge(minutesWaited);
  return { fare, surcharge, waitingCharge: waiting, total: fare + surcharge + waiting };
}


export function totalDistance(rides) {
  let sum = 0;
  for (const ride of rides) {
    sum += ride.distanceKm;
  }
  return sum;
}
```

Appendix A2: package.json

```
  "name": "lab04",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "directories": {
    "test": "test"
  },
  "scripts": {
    "test": "node --test"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "module"
}
```

Appendix A3: test/taxi.test.js

```
import { describe, it } from 'node:test';
```

```
import assert from 'node:assert/strict';
import {
  fareForRide, totalFare, applyPeakSurcharge,
  waitingCharge, rideSummary, totalDistance,
} from '../src/taxi.js';

describe('R1 fareForRide', () => {
  it('multiplies distance by rate for whole numbers', () => {
    // Arrange
    const km = 10, rate = 30;
    // Act
    const result = fareForRide(km, rate);
    // Assert: 10 x 30 = 300
    assert.strictEqual(result, 300);
  });

  it('rounds a .5 fare up to the nearest rupee', () => {
    const result = fareForRide(7.5, 25);
    // 7.5 x 25 = 187.5 -> 188
    assert.strictEqual(result, 188);
  });

  it('rounds a fractional fare down when below .5', () => {
    const result = fareForRide(3.3, 10);
    // 3.3 x 10 = 33 -> 33; use 2.2 x 11 = 24.2 -> 24
    assert.strictEqual(fareForRide(2.2, 11), 24);
    assert.strictEqual(result, 33);
  });

  it('returns 0 for a zero-distance ride', () => {
    assert.strictEqual(fareForRide(0, 30), 0);
  });
});

describe('R2 totalFare', () => {
  it('adds the fares of several rides', () => {
    const rides = [
      { distanceKm: 10, ratePerKm: 30 },
      { distanceKm: 5, ratePerKm: 40 },
    ];
    // 300 + 200 = 500
    assert.strictEqual(totalFare(rides), 500);
  });

  it('returns 0 for an empty list of rides', () => {
    assert.strictEqual(totalFare([]), 0);
  });

  it('rounds each ride separately before adding (per R1)', () => {
    const rides = [
      { distanceKm: 2.5, ratePerKm: 25 },
      { distanceKm: 2.5, ratePerKm: 25 },
    ];
    // 62.5 -> 63 each, 63 + 63 = 126
    assert.strictEqual(totalFare(rides), 126);
  });
});
```

```
describe('R3 applyPeakSurcharge', () => {
  it('increases the amount by 20 percent', () => {
    // 1000 + 20% = 1200
    assert.strictEqual(applyPeakSurcharge(1000, 20), 1200);
  });

  it('rounds the surcharged amount to the nearest rupee', () => {
    // 333 x 1.10 = 366.3 -> 366
    assert.strictEqual(applyPeakSurcharge(333, 10), 366);
  });

  it('leaves the amount unchanged at 0 percent', () => {
    assert.strictEqual(applyPeakSurcharge(450, 0), 450);
  });
});

describe('R4 waitingCharge', () => {
  it('is free at 4 minutes (below the free limit)', () => {
    assert.strictEqual(waitingCharge(4), 0);
  });

  it('is free at exactly 5 minutes (boundary)', () => {
    assert.strictEqual(waitingCharge(5), 0);
  });


  it('costs Rs 10 at 6 minutes (one minute beyond the free limit)', () => {
    assert.strictEqual(waitingCharge(6), 10);
  });


  it('charges Rs 10 for each minute beyond 5 for a long wait', () => {
    // 15 - 5 = 10 minutes x 10 = 100
    assert.strictEqual(waitingCharge(15), 100);
  });
});


describe('R5 rideSummary', () => {
  it('returns fare, surcharge, waitingCharge and total', () => {
    const rides = [
      { distanceKm: 10, ratePerKm: 30 },
      { distanceKm: 5, ratePerKm: 40 },
    ];
    const result = rideSummary(rides, 10, 8);
    // fare 500; surcharge 550-500 = 50; waiting (8-5) x 10 = 30; total 580
    assert.deepStrictEqual(result, { fare: 500, surcharge: 50, waitingCharge: 30, total: 580 });
  });


  it('returns all zeros for no rides, 0% surcharge and no waiting', () => {
    assert.deepStrictEqual(rideSummary([], 0, 0),
      { fare: 0, surcharge: 0, waitingCharge: 0, total: 0 });
  });
});


describe('R6 totalDistance', () => {
  it('sums distanceKm across all rides', () => {
```

```
    const rides = [{ distanceKm: 10 }, { distanceKm: 5.5 }, { distanceKm: 4.5 }];
    // 10 + 5.5 + 4.5 = 20
    assert.strictEqual(totalDistance(rides), 20);
  });

  it('returns 0 for an empty list of rides', () => {
    assert.strictEqual(totalDistance([]), 0);
  });

  it('returns the distance of a single ride', () => {
    assert.strictEqual(totalDistance([{ distanceKm: 7 }]), 7);
  });
});
```